

Rapport d'utilisation de la librairie JavaScript *mmenu-light* pour la gestion du menu burger en mode téléphone et tablette

1. Besoin

L'intitulé de notre projet SAE de développement d'un site web demande la prise en charge d'une version mobile avec un menu burger. Ce type de menu nécessite l'utilisation d'un script javascript qui permet la modification, sans rechargement, de l'affichage de la page.

Il était important pour nous de trouver une librairie minimale, permettant uniquement la prise en charge du menu burger afin de limiter la complexité de son utilisation et le poids ajouté par celle-ci.

Il fallait également trouver une librairie qui n'utilise que du javascript et des feuilles de style css.

La librairie devait aussi être compatible avec les pages html et css déjà existante de notre site et ne pas imposer l'utilisation de balisage complexe.

Enfin, il était important de trouver une librairie documentée, pour faciliter sa compréhension et son intégration.

2. Choix de la librairie

Nous avons opté pour l'utilisation de la librairie ***mmenu-light.js*** très bien documentée à l'adresse suivante : <https://www.mmenujs.com/mmenu-light/>

Cette librairie satisfait à tous les besoins recensés et permet en plus l'affichage d'un menu offcanvas (panneau latéral qui apparaît en glissant depuis la gauche) qui correspond aux standards actuels des affichages de menu burger sur mobile.

L'intégration de la librairie dans notre projet a nécessité le téléchargement de 2 fichiers : ***mmenu-light.js*** et ***mmenu-light.css*** disponibles sur github à l'adresse suivante : <https://github.com/FrDH/mmenu-light/tree/master/dist>

Le poids total des 2 fichiers cumulé fait seulement 7ko ce qui la rend compatible avec le mode éco-responsable de notre site.

3. Mise en place de la dépendance

Les 2 fichiers sources de la librairie ont été intégrés aux fichiers de notre projet :

- ***mmenu-light.css*** dans le répertoire `src/css/`
- ***mmenu-light.js*** dans le répertoire `src/js`

Le chargement des fichiers se fait en 2 temps dans chacune de nos pages html :

- `<link rel="stylesheet" href="src/css/mmenu-light.css">` dans la balise `<head>` pour permettre le chargement de la feuille de style ***mmenu-light.css***
- `<script src="src/js/mmenu-light.js"></script>` en fin de fichier, juste avant la balise de fermeture du body `</body>` afin d'être certain que la page soit chargée avant le script.

Enfin, l'utilisation de cette librairie nécessite la création d'un petit script javascript (documenté dans le site ***mmenujs***). Nous avons donc créé un fichier ***init-mmenu-light.js*** dans le répertoire `src/js`. Ce script est appelé juste après le script de la librairie dans nos fichier html. Ce qui donne, juste avant la balise `</body>` :

Dans les fichiers html du site :

```
<script src="src/js/mmenu-light.js"></script>
<script src="src/js/init-mmenu-light.js"></script>
</body>
```

4. Fonctionnement de la librairie

La librairie nécessite une initialisation pour définir son comportement et les composant html de la page avec lesquelles elle doit interagir. C'est le rôle du script ***init-mmenu-light.js***.

Nous allons détailler ici bloc par bloc, le but de chaque ligne du script ***init-mmenu-light.js***.

4.1. Premier bloc

Dans le fichier `init-mmenu-light.js` :

```
const menu = new MmenuLight(
  document.querySelector('#menu-mobile')
);
```

On initialise ici la librairie en lui indiquant quel est l'identifiant (id) de l'élément HTML qui représente notre menu mobile : ***#menu-mobile***

Ainsi on indique à la librairie que le contenu du menu mobile se trouve dans la balise nav suivante :

Dans les fichiers html du site :

```
<nav id="menu-mobile" aria-label="Menu principal mobile">
  <ul>
    <li><a href="index.html">Accueil</a></li>
    <li><a href="produits.html">Nos produits</a></li>
    <li><a href="temoignages.html">Témoignages clients</a></li>
    <li><a href="equipe.html">Notre équipe</a></li>
    <li><a href="contact.html">Nous contacter</a></li>
  </ul>
</nav>
```

Ce contenu est représenté sous forme de liste avec des balises **** et ****.

4.2. Second bloc

Dans le fichier init-mmenu-light.js :

```
const navigator = menu.navigation({
  selectedClass: 'selected',
  slidingSubmenus: true,
  title: 'Menu'
});
```

Ici nous déterminons comment nous souhaitons que la librairie affiche notre menu en définissant 3 options :

- **selectedClass** : permet d'indiquer à l'utilisateur où il est. On ajoute donc dans chaque page la **class= « selected »** sur l'élément **** correspondant à la page en cours.

Exemple pour indiquer Accueil :

```
<nav id="menu-mobile" aria-label="Menu principal mobile">
  <ul>
    <li class="selected"><a href="index.html">Accueil</a></li>
    ...
```

- **slidingSubmenus** : permet d'activer la navigation par panneau.
- **title** : permet de définir le titre qui doit être affiché en haut du menu.

Ici la librairie agit directement sur notre page html en créant le menu glissant à partir de la feuille css de la librairie **mmenu-light.css**.

4.3. Troisième bloc

Dans le fichier init-mmenu-light.js :

```
const drawer = menu.offcanvas();
```

Ceci permet l'activation du menu `offcanvas` : le menu est masqué et s'affiche en glissant depuis la gauche de l'écran. C'est la librairie qui gère l'ouverture et la fermeture du menu et les animations grâce à sa feuille de style ***mmenu-light.css***.

4.4. Dernier bloc

Dans le fichier `init-mmenu-light.js` :

```
document
  .querySelector('a[href="#menu-mobile"]')
  .addEventListener('click', (ev) => {
    ev.preventDefault();
    drawer.open();
  });
```

Ici le but est de créer un `EventListener` qui permet d'intercepter le clic sur le menu burger et d'appeler la méthode ***open()*** de la librairie qui va déclencher l'affichage du menu suivant les paramètres définis précédemment.

.querySelector('a[href="#menu-mobile"]') permet d'indiquer où se trouve le menu burger sur lequel l'évènement clic doit être intercepté. Le script cherche un lien ***<a>*** dont l'attribut ***href*** est égale à ***#menu-mobile***.

Ceci correspond dans nos pages html à la représentation du menu burger :

```
<a href="#menu-mobile" class="burger" aria-label="Ouvrir le menu" aria-
controls="menu-mobile" aria-expanded="false">
  <span></span>
  <span></span>
  <span></span>
</a>
```

5. Conclusion

L'utilisation de la librairie ***mmenu-light.js*** nous permet facilement et avec un impact minimal sur le poids de nos pages de créer un menu burger sobre et moderne. Son intégration dans nos pages html n'a nécessité que 2 nouveaux blocs très simples pour décrire le contenu du menu et le lien permettant son ouverture. Le script d'initialisation a demandé peu d'effort puisqu'il était fourni dans la documentation et les exemples.

Rapport d'utilisation du fichier JavaScript *accordion* pour la gestion de l'accordéon de la FAQ

1. Besoin

Il est recommandé d'inclure un menu accordéon pour la section FAQ du site. Il est nécessaire d'utiliser du JavaScript pour réaliser ce genre de menu accordéon, où cliquer sur un élément permet de faire dérouler et apparaître une zone de texte.

Il nous faut un fichier JavaScript assez léger pour pouvoir être intégré à notre site tout en respectant nos valeurs écoresponsables.

Enfin, il fallait que le fichier JavaScript soit compatible avec notre site, sans avoir à modifier complètement la structure et le balisage de nos fichiers HTML et CSS.

2. Choix de la librairie

Nous avons opté pour l'utilisation de ce code JavaScript présent et documenté à l'adresse suivante : https://www.w3schools.com/howto/howto_js_accordion.asp

Le fichier accordion.js est très simple et compatible avec la structure HTML de notre site avec quelques petits changements : il a fallu remplacer les balises `<p class="question">` par des balises `<button class="question">` et il a fallu mettre les balises `<p class="reponse">` dans des balise `<div>` et retirer la dite class pour la donner aux balises div. Le CSS a également eu quelques ajouts détaillés ci-après garder une mise en page similaire.

Sa simplicité le rend très léger, il ne pèse que 1ko, ce qui le rend compatible avec le mode écoresponsable de notre site.

3. Mise en place de la dépendance

Le fichier accordion.js est intégré dans le répertoire src/js

Le chargement de ce fichier se fait dans nos pages contact.html et contact-eco.html dans la balise <head> avec la balise suivante :

<script src= "src/js/accordion.js" defer></script>. L'attribut **defer** permet de charger le JavaScript une fois que la page HTML est chargée.

4. Fonctionnement de la librairie

Nous allons ici détailler le fonctionnement du fichier accordion.js, bloc par bloc.

4.1. Premier bloc

```
var acc = document.getElementsByClassName("question");  
var i;
```

On initialise d'abord deux variables :

-**acc** correspond aux éléments du document HTML dont la classe est "**question**", donc, aux boutons comportant les questions.

-**i** est un simple compteur.

4.2. Deuxième bloc

```
for (i = 0; i < acc.length; i++) {  
  acc[i].addEventListener("click", function() {  
    this.classList.toggle("active");  
    var panel = this.nextElementSibling;  
    if (panel.style.display === "block") {  
      panel.style.display = "none";  
    } else {  
      panel.style.display = "block";  
    }  
  })  
}
```

L. 1-3 Sur toute la longueur de acc (les boutons de classe **"question"**), cliquer dessus va rajouter à l'élément la classe **"active"**.

L. 4 On définit la variable **"panel"** qui correspond aux éléments « frères » des éléments acc, donc les balises `<div class="reponse">`

L.5-9 Si le display des élément **"panel"** dans le fichier CSS est **"block"**, au moment où l'on clique sur le bouton, on remet leur display à **"none"**, sinon, on le met à **"block"**.

Comme le display de base de **"panel"** est **"none"**, le texte n'est donc pas visible. cliquer sur le bouton va donner la classe **"active"** au bouton et le display **"block"** aux **"panel"**, ce qui affiche le texte.

4.3. Dernier bloc

```
if (panel.style.maxHeight) {  
    panel.style.maxHeight = null;  
} else {  
    panel.style.maxHeight = panel.scrollHeight + "px";  
}
```

Cette partie du code permet de faire une transition quand on clique sur un bouton : Si le texte de **"panel"** à sa taille maximum, donc si le texte est visible, on lui donne une taille null, sinon on augmente petit à petit sa taille d'un pixel jusqu'à ce que **"panel"** atteigne sa taille maximale.

4.4. Ajouts au fichier CSS

```
.question {  
    text-align: left;  
    cursor: pointer;  
    transition: all 0.3s ease;  
    ...  
}
```

Rajouter ceci aux boutons de classe **"question"** permet d'aligner le texte à gauche pour avoir la même mise en page que dans nos versions précédentes, de changer le curseur de la souris par un pointeur quand on survole le bouton, pour indiquer qu'il est cliquable et de mettre une transition simple et non brusque pour le changement de couleur induit par :

```
.active, .question:hover {  
    background-color: #ccc;  
}
```

Cette partie du CSS fait changer la couleur des boutons quand ils sont **"active"** ou qu'ils sont survolés par la souris.

```
.question:after {  
    content: "+";  
    font-size: 2rem;  
    color: var(--couleur-generale);  
    float: right;  
    margin-left: 0.5rem;  
}  
  
.active:after {  
    content: "-";  
}
```

Cette partie du CSS rajoute un « + » à la droite du bouton pour indiquer que cliquer dessus fera apparaître du texte. Une fois le bouton **"active"**, on change le « + » pour un « - », ce qui indique que re-cliquer sur le bouton fera disparaître le texte.

```
.reponse {  
    display: none;
```



```
overflow: hidden;

margin: 1.2rem 2rem 2.5rem 2rem;

font-size: 1.2rem;

max-height: 0;

transition: max-height 0.3s ease-out;

}
```

Cette partie du CSS permet de rendre le texte correspondant à la réponse invisible. Son display est initialisé à **"none"** pour qu'il ne soit pas visible et son overflow est mis à **"hidden"** pour que le texte apparaisse progressivement lors de la transition. Si on le retire, le texte apparaît dès qu'il a assez d'espace. Enfin, transition permet de bien faire apparaître le texte proprement lors de la transition.

5. Conclusion

L'utilisation de **accordion.js** nous permet de créer sans trop alourdir le site un menu accordéon simple et efficace. Son intégration est simple et n'a pas demandé de changer énormément nos pages contact.